
Bölüm-3

SINIFLARA VE NESNELERE DERİN BİR BAKIŞ



Bu bölümde;

- ▶ Bir sınıf tanımlamanın detaylarını göreceksiniz.
- ▶ Bir problem ile karşılaşıldığında “throw” ifadesini kullanacaksınız.
- ▶ Aynı sınıftaki bir constructor içinde “this” anahtar kelimesi ile başka bir constructor çağırılmasını kullanacaksınız.
- ▶ Composition kavramını öğreneceksiniz.
- ▶ Static değişkenler ve metodları kullanacaksınız.
- ▶ Static üyelerin import edeceksiniz.
- ▶ Tekil tanımlayıcılarla sabitler yaratmak için “enum” kullanacaksınız.
- ▶ Parametreler ile “enum” sabitleri tanımlayacaksınız.
- ▶ Parasal hesaplamalar için “BigDecimal” kullanacaksınız.



- ▶ Giriş
- ▶ Time Class Case Study
- ▶ Üyelere Erişimin Kontrol Edilmesi
- ▶ Nesnenin üyelerine “this” ile erişme
- ▶ Constructor overloading
- ▶ Composition
- ▶ “final” örnek değişkenleri
- ▶ Paket Erişimleri
- ▶ Parasal hesaplamalar için “BigDecimal” kullanımı
- ▶ CASE STUDY: Nesneleri GUI ile kullanma



- ▶ Derin bir bakış ile nesnelere ve üyelere erişimin kontrol edilmesi.
- ▶ Hatalı durumlara karşı “throw” istisnalarının kullanması.
- ▶ Bir sınıfın nesneleri ile diğer sınıfın üyelerine referans verme izni.
- ▶ Enum hakkında detaylar
- ▶ “static” ve “final” değişkenler
- ▶ Büyük uygulamalarda paket kullanımı



- ▶ Günün zamanlarını gösterecek bir “Time1” sınıfı oluşturalım.
- ▶ Sınıfının “private” olarak hour, minute, second değişkenleri olsun.
 - ▶ 24 saatlik dilim kullanılsın. Hour (0-23), minute ve second (0-59) aralığında olmalı.
- ▶ Public metotlar: setTime(), toUniversalString() ve toString().

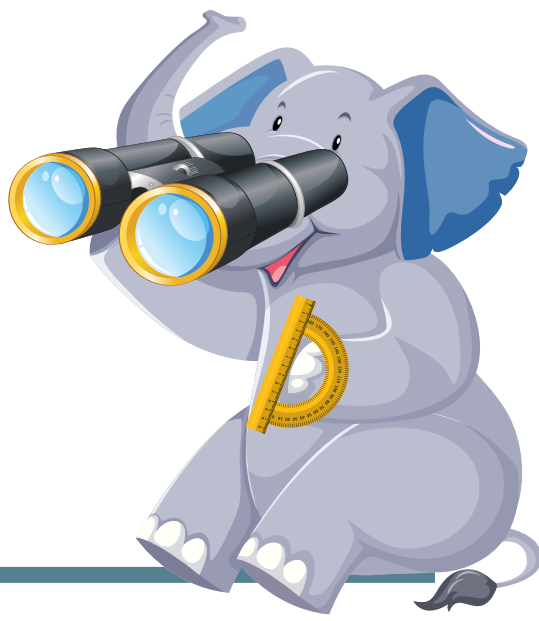


- ▶ Sınıfa ait değişkenler tanımlandı.
- ▶ Setter metodu içinde geçerli değerler girilmesi kontrol edilmeli. ("Throw" ifadesi)
- ▶ Eğer değerler geçerli aralık dışında ise setTime metodu bir istisna atar (throw an exception).
- ▶ İstisna türü: "IllegalArgumentException"
- ▶ IllegalArgumentException tipinde bir nesne create edilir.
- ▶ Default mesaj da verilebilir.
- ▶ Custom bir mesaj da yazılabilir.

```
1 // Fig. 8.1: Time1.java
2 // Time1 class declaration maintains the time in 24-hour format.
3
4 public class Time1 {
5     private int hour; // 0 - 23
6     private int minute; // 0 - 59
7     private int second; // 0 - 59
8
9     // set a new time value using universal time; throw an
10    // exception if the hour, minute or second is invalid
11    public void setTime(int hour, int minute, int second) {
12        // validate hour, minute and second
13        if (hour < 0 || hour >= 24 || minute < 0 || minute >= 60 ||
14            second < 0 || second >= 60) {
15            throw new IllegalArgumentException(
16                "hour, minute and/or second was out of range");
17        }
18
19        this.hour = hour;
20        this.minute = minute;
21        this.second = second;
22    }
```

Fig. 8.1 | Time1 class declaration maintains the time in 24-hour format. (Part 1 of 2.)

SOFTWARE ENGINEERING OBSERVATION



setTimeout() gibi metotlarda, metodun tüm argümanlarını kullanmadan önce doğrulayın.



- ▶ `toUniversalString()` metodu, evrensel formatta bir saat gösterimi sunar.
- ▶ Tüm değişkenler iki basamak halinde formatlanır. (2d)
- ▶ `toString()` yöntemi ise 12lik dilimde sonuçları verir.
 - ▶ A.M. (öğleden önce)
 - ▶ P.M. (öğleden sonra)
 - ▶ Ternary operator kullanılmış.
 - ▶ **KOŞUL ? True_statement : false_statement**

```
23
24 // convert to String in universal-time format (HH:MM:SS)
25 public String toUniversalString() {
26     return String.format("%02d:%02d:%02d", hour, minute, second);
27 }
28
29 // convert to String in standard-time format (H:MM:SS AM or PM)
30 public String toString() {
31     return String.format("%d:%02d:%02d %s",
32         ((hour == 0 || hour == 12) ? 12 : hour % 12),
33         minute, second, (hour < 12 ? "AM" : "PM"));
34 }
35 }
```

Fig. 8.1 | Time1 class declaration maintains the time in 24-hour format. (Part 2 of 2.)



- ▶ TimeTest sınıfı test için tanımlanmıştır.
- ▶ Main metodu içinde bir Time1 nesnesi yaratılır.
- ▶ DisplayTime methodu ile zaman ekrana farklı formatlarda basılır.
- ▶ Bu yöntem sadece bu sınıfta kullanılacağı (yardımcı yöntem) için private static olarak tanımlanmasında sakınca yoktur.
- ▶ Böylece nesneye gerek duymadan doğrudan çağırabildik.

```
1 // Fig. 8.2: Time1Test.java
2 // Time1 object used in an app.
3
4 public class Time1Test {
5     public static void main(String[] args) {
6         // create and initialize a Time1 object
7         Time1 time = new Time1(); // invokes Time1 constructor
8
9         // output string representations of the time
10        displayTime("After time object is created", time);
11        System.out.println();
12
13        // change time and output updated time
14        time.setTime(13, 27, 6);
15        displayTime("After calling setTime", time);
16        System.out.println();
17    }
```

Fig. 8.2 | Time1 object used in an app. (Part I of 3.)



- ▶ TimeTest sınıfı test için tanımlanmıştır.
- ▶ Main metodu içinde bir Time1 nesnesi yaratılır.
- ▶ DisplayTime methodu ile zaman ekrana farklı formatlarda basılır.
- ▶ Bu yöntem sadece bu sınıfta kullanılacağı (yardımcı yöntem) için private static olarak tanımlanmasında sakınca yoktur.
- ▶ Böylece nesneye gerek duymadan doğrudan çağırabildik.

```
1 // Fig. 8.2: Time1Test.java
2 // Time1 object used in an app.
```

```
18 // attempt to set time with invalid values
19 try {
20     time.setTime(99, 99, 99); // all values out of range
21 }
22 catch (IllegalArgumentException e) {
23     System.out.printf("Exception: %s\n\n", e.getMessage());
24 }
25
26 // display time after attempt to set invalid values
27 displayTime("After calling setTime with invalid values", time);
28 }
29
30 // displays a Time1 object in 24-hour and 12-hour formats
31 private static void displayTime(String header, Time1 t) {
32     System.out.printf("%s\nUniversal time: %s\nStandard time: %s\n",
33         header, t.toUniversalString(), t.toString());
34 }
35 }
```

Fig. 8.2 | Time1 object used in an app. (Part 2 of 3.)



- ▶ `toString()` metodunda saat değeri için “%d” olduğundan başına sıfır (0) eklenmedi.
- ▶ Hatalı değer girildiğinde try bloğundan olumsuz sonuç gelecek, ve bir `IllegalArgumentException` atılacaktır.
- ▶ Bu istisna. `e.getMessage()` metodunu çağırarak ilgili hata mesajını ekrana basacaktır.

```
After time object is created
Universal time: 00:00:00
Standard time: 12:00:00 AM

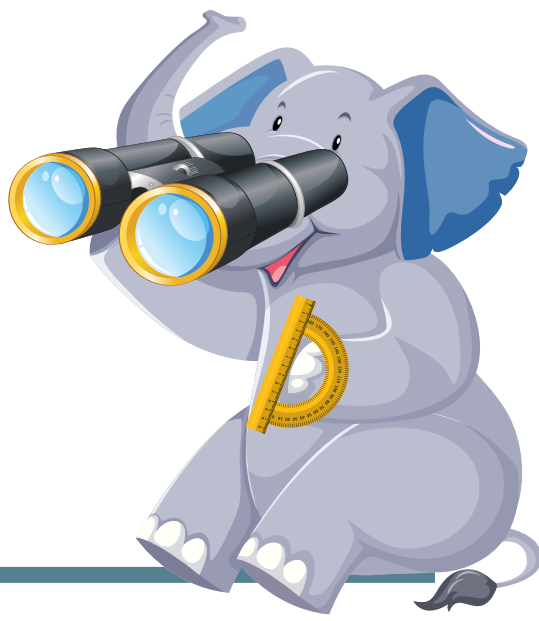
After calling setTime
Universal time: 13:27:06
Standard time: 1:27:06 PM

Exception: hour, minute and/or second was out of range

After calling setTime with invalid values
Universal time: 13:27:06
Standard time: 1:27:06 PM
```

Fig. 8.2 | Time1 object used in an app. (Part 3 of 3.)

SOFTWARE ENGINEERING OBSERVATION



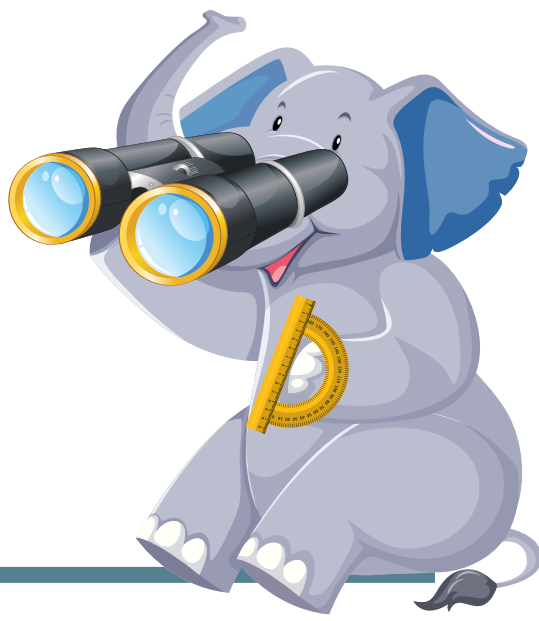
Private metotlara sadece o metotların tanımlandığı sınıftaki metodlar erişebilir.

Bu yüzden bu metotlara diğer metotların bir işlevini gerçekleştirdikleri için “helper/utility method” ismi verilir.



- ▶ Time1 sınıfındaki değişkenler private olarak verilmiştir.
- ▶ Bu durum, bu sınıfın client larını etkilemez.
- ▶ Eğer bu veriler başka bir amaçla kullanılacaksa, yine bir public method aracılığı ile kullanılabilir.
- ▶

SOFTWARE ENGINEERING OBSERVATION



Sınıflarda metotların “public” olarak tanımlanması client tarafından işleri kolaylaştırır. Client sınıf içinde işlerin nasıl kodlandığının farkında değildir.

İşin nasıl yapılacağına değil ne yapabileceğine odaklanır.



- ▶ Tarih ve Zaman bilgisi için, sınıflar yaratmak yerine java API kullanılabilir.
- ▶ JAVA8 ile birlikte, "java.time" paketi altında bir "date/time API" tanıtılmıştır.
- ▶ Bir ÖRNEK;

```
import java.time.LocalDateTime;  
  
public class CurrentDateTimeExample {  
    public static void main(String[] args) {  
        // Current Date and Time  
        LocalDateTime currentDateTime = LocalDateTime.now();  
        System.out.println("Current Date & time: "  
            + currentDateTime);  
    }  
}
```





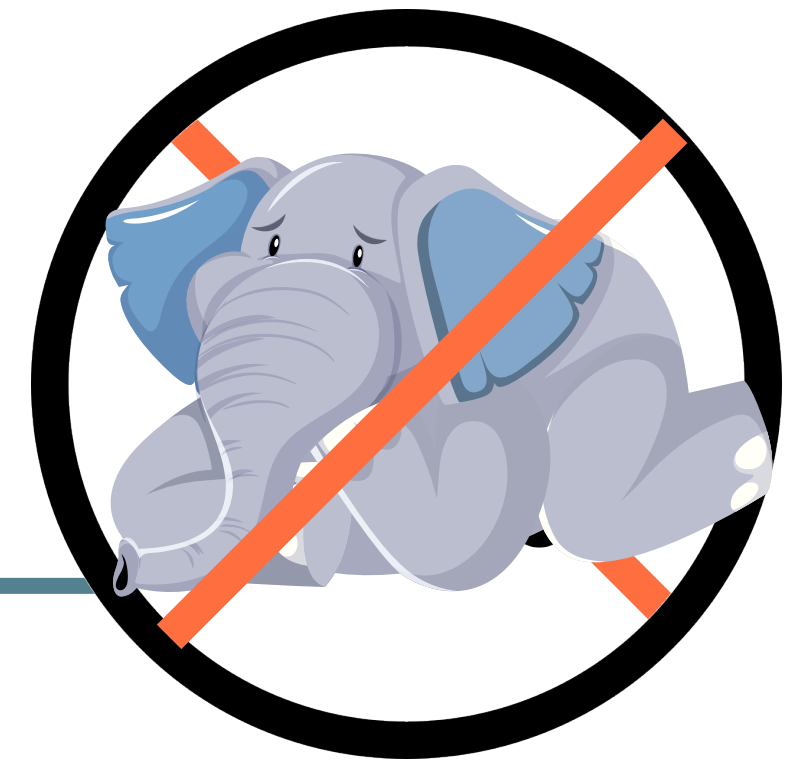
- ▶ Değişken ve metotlara erişim için iki denetleyici bulunmaktadır. (Şimdiye kadar gördüklerimiz)
- ▶ `private` ve `public`
- ▶ `public` metotlar client hizmetine sunulmuştur.
- ▶ `private` değişkenlere sınıf dışından erişim yetkisi yoktur.

```
1 // Fig. 8.3: MemberAccessTest.java
2 // Private members of class Time1 are not accessible.
3 public class MemberAccessTest {
4     public static void main(String[] args) {
5         Time1 time = new Time1(); // create and initialize Time1 object
6
7         time.hour = 7; // error: hour has private access in Time1
8         time.minute = 15; // error: minute has private access in Time1
9         time.second = 30; // error: second has private access in Time1
10    }
11 }
```

```
MemberAccessTest.java:7: error: hour has private access in Time1
    time.hour = 7; // error: hour has private access in Time1
        ^
MemberAccessTest.java:8: error: minute has private access in Time1
    time.minute = 15; // error: minute has private access in Time1
        ^
MemberAccessTest.java:9: error: second has private access in Time1
    time.second = 30; // error: second has private access in Time1
        ^
3 errors
```

Fig. 8.3 | Private members of class `Time1` are not accessible.

COMMON PROGRAMMING ERROR



Sınıfın üyesi olmayan bir metot tarafından private üyelere erişmeyi denemek derleme hatasına yol açacaktır.



- ▶ Her nesne kendisine “this” anahtar kelimesi ile referans vererek erişebilir.
- ▶ Bir metod çağırıldığında, metod gövdesinde nesnenin değişkenlerine ve metotlarına üstü kapalı biçimde (implicitly) “this” anahtar kelimesi ile referans verir.
- ▶ Metotlar açık bir şekilde (explicitly) “this” anahtar kelimesini de kullanabilir.
- ▶ Örnek olarak buildString (Fig.8.4) metodu iki farklı çağırımı da göstermektedir.
 - ▶ Burada metod ismini gölgeleyecek bir yerel değişken olmadığından “this” kullanımı gerekli değildir.



```

1 // Fig. 8.4: ThisTest.java
2 // this used implicitly and explicitly to refer to members of an object.
3
4 public class ThisTest {
5     public static void main(String[] args) {
6         SimpleTime time = new SimpleTime(15, 30, 19);
7         System.out.println(time.buildString());
8     }
9 }
10
11 // class SimpleTime demonstrates the "this" reference
12 class SimpleTime {
13     private int hour; // 0-23
14     private int minute; // 0-59
15     private int second; // 0-59
16
17     // if the constructor uses parameter names identical to
18     // instance variable names the "this" reference is
19     // required to distinguish between the names
20     public SimpleTime(int hour, int minute, int second) {
21         this.hour = hour; // set "this" object's hour
22         this.minute = minute; // set "this" object's minute
23         this.second = second; // set "this" object's second
24     }
25

```

```

26 // use explicit and implicit "this" to call toUniversalString
27 public String buildString() {
28     return String.format("%24s: %s\n%24s: %s",
29         "this.toUniversalString()", this.toUniversalString(),
30         "toUniversalString()", toUniversalString());
31 }
32
33 // convert to String in universal-time format (HH:MM:SS)
34 public String toUniversalString() {
35     // "this" is not required here to access instance variables,
36     // because method does not have local variables with same
37     // names as instance variables
38     return String.format("%02d:%02d:%02d",
39         this.hour, this.minute, this.second);
40 }
41

```

```

this.toUniversalString(): 15:30:19
toUniversalString(): 15:30:19

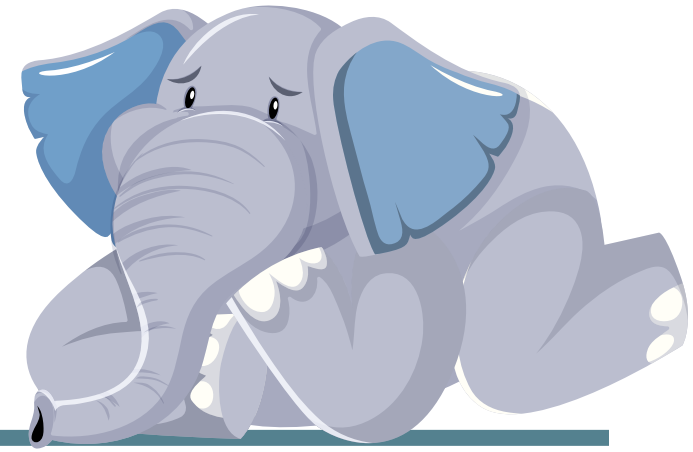
```

Fig. 8.4 | this used implicitly and explicitly to refer to members of an object. (Part 3 of 3.)



- ▶ Örnekte görüldüğü gibi (Fig.8.4) birden fazla sınıf barındıran bir .java dosyası derlendiğinde;
 - ▶ Derleyici derlenen sınıflar için ayrı ayrı .class dosyaları oluşturur.
 - ▶ Bu dosyalar aynı dizine yerleştirilir.
 - ▶ Bu sınıflardan sadece bir tanesi “public” olmalıdır. Aksi takdirde derleme hatası oluşur.
 - ▶ Public olmayan sınıfı sadece aynı pakette bulunan diğer sınıflar kullanabilir.
- ▶ SimpleTime sınıfı 3 private örnek değişken bildirir. (Hour, minute ve second)
- ▶ Eğer sınıfa ait constructor parametre adları ile örnek değişken adları aynı ise; örnek değişkenlere referans vermek için “this” kullanılır.

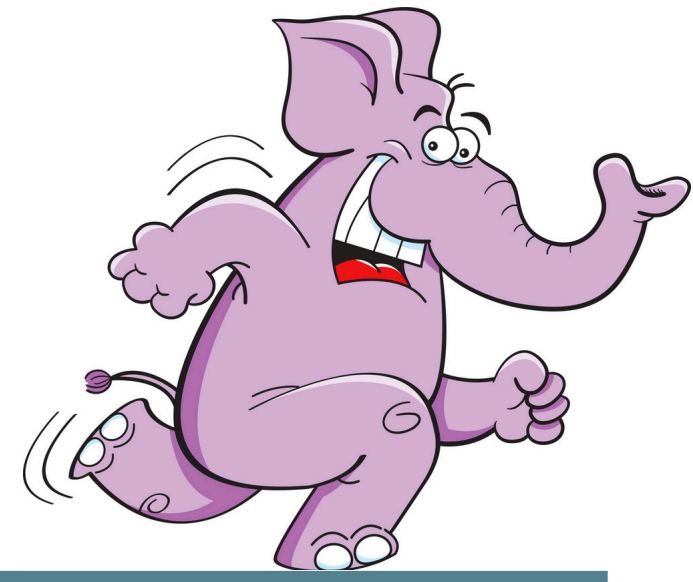
ERROR PREVENTION TIP



Çoğu derleyici; “`this.x = x`” yerine, “`x = x`” kullanıldığında bu ifade için bir uyarı verir.

“`x = x`” ifadesi genellikle işlemsiz (no op) olarak adlandırılır.

PERFORMANCE TIP



Sınıf başına her yöntemin yalnızca bir kopyası vardır; sınıfın her nesnesi, yöntemin kodunu paylaşır.

Öte yandan her nesne, sınıfın örnek değişkenlerinin kendi kopyasına sahiptir.

Sınıfın statik olmayan yöntemleri, sınıfın manipüle edilecek nesnesini belirlemek için “this” anahtar kelimesini “implicitly” kullanır.



- ▶ Constructor overloading; sınıfın nesnelerini farklı yollardan ilklendirmeyi sağlar.
- ▶ Farklı imzalara sahip çoklu constructor ile kolaylıkla bu işlem yapılabilir.
- ▶ Derleyici;
 - ▶ parametre sayısı
 - ▶ parametre tipi
 - ▶ parametrelerin sırasına göre imzaları ayırt eder.
- ▶ Derleyici tarafından, constructor çağrısındaki parametrelere göre uygun constructor çağrılır.
- ▶ Başka bir constructor çağıran constructor; “delegate (temsilci) constructor” olarak bilinir.
- ▶ Sınıfın diğer constructor ları tarafından sağlanan ilklendirme kodunu yeniden kullanmanın en popüler yolu “this” kullanımıdır.



- ▶ Time2 sınıfında, nesneleri ilklendirmek için uygun yollar sağlayan 5 overloaded constructor tanımlanmıştır.
- ▶ İlk constructor argümentsiz olarak tanımlanmıştır.
- ▶ 3-parametrelili constructor çağırılır.
- ▶ Default değerler atanmış olur.
- ▶ Tek parametrelili constructor saat değerini alır, diğer iki değişken yine 0 olarak belirlenir.
- ▶ Benzer şekilde 2-li constructor tanımlanmıştır.

```
1 // Fig. 8.5: Time2.java
2 // Time2 class declaration with overloaded constructors.
3
4 public class Time2 {
5     private int hour; // 0 - 23
6     private int minute; // 0 - 59
7     private int second; // 0 - 59
8
9     // Time2 no-argument constructor:
10    // initializes each instance variable to zero
11    public Time2() {
12        this(0, 0, 0); // invoke constructor with three arguments
13    }
14
15    // Time2 constructor: hour supplied, minute and second defaulted to 0
16    public Time2(int hour) {
17        this(hour, 0, 0); // invoke constructor with three arguments
18    }
19
20    // Time2 constructor: hour and minute supplied, second defaulted to 0
21    public Time2(int hour, int minute) {
22        this(hour, minute, 0); // invoke constructor with three arguments
23    }
24 }
```

Fig. 8.5 | Time2 class declaration with overloaded constructors. (Part I of 6.)



- ▶ 3-lü constructor tüm değişkenleri ilklendirir.
- ▶ Bir diğer constructor ise başka bir Time2 nesnesi çağırır. Bu nesneye ait değerler 3-lü constructor ile ilklendirilir.
- ▶ Burada constructor parametresi olan "time" nesnesinin değerleri yeni nesneye aktarılır. (??)
- ▶ 4 farklı constructor, tek bir tanesine yönlendirilmiştir. Böylece, benzer kodları tekrar yazmak yerine, ilklendirme kodunun yeniden kullanımı sağlanmış olur.
- ▶ Eğer kullanım ile alakalı bir değişiklik yapılacaksa, sadece 3-lü constructor yeniden yazılır/düzenlenir.
- ▶ Böylece, düzenlemesi ve sürdürmesi daha kolay bir yapı sağlanmış olur.

```
25 // Time2 constructor: hour, minute and second supplied
26 public Time2(int hour, int minute, int second) {
27     if (hour < 0 || hour >= 24) {
28         throw new IllegalArgumentException("hour must be 0-23");
29     }
30
31     if (minute < 0 || minute >= 60) {
32         throw new IllegalArgumentException("minute must be 0-59");
33     }
34
35     if (second < 0 || second >= 60) {
36         throw new IllegalArgumentException("second must be 0-59");
37     }
38
39     this.hour = hour;
40     this.minute = minute;
41     this.second = second;
42 }
43
44 // Time2 constructor: another Time2 object supplied
45 public Time2(Time2 time) {
46     // invoke constructor with three arguments
47     this(time.hour, time.minute, time.second);
48 }
```

Fig. 8.5 | Time2 class declaration with overloaded constructors. (Part 2 of 6.)



```
49
50 // Set Methods
51 // set a new time value using universal time;
52 // validate the data
53 public void setTime(int hour, int minute, int second) {
54     if (hour < 0 || hour >= 24) {
55         throw new IllegalArgumentException("hour must be 0-23");
56     }
57
58     if (minute < 0 || minute >= 60) {
59         throw new IllegalArgumentException("minute must be 0-59");
60     }
61
62     if (second < 0 || second >= 60) {
63         throw new IllegalArgumentException("second must be 0-59");
64     }
65
66     this.hour = hour;
67     this.minute = minute;
68     this.second = second;
69 }
```

Fig. 8.5 | Time2 class declaration with overloaded constructors. (Part 3 of 6.)



```
70
71 // validate and set hour
72 public void setHour(int hour) {
73     if (hour < 0 || hour >= 24) {
74         throw new IllegalArgumentException("hour must be 0-23");
75     }
76
77     this.hour = hour;
78 }
79
80 // validate and set minute
81 public void setMinute(int minute) {
82     if (minute < 0 || minute >= 60) {
83         throw new IllegalArgumentException("minute must be 0-59");
84     }
85
86     this.minute = minute;
87 }
88 }
```

Fig. 8.5 | Time2 class declaration with overloaded constructors. (Part 4 of 6.)



```

89 // validate and set second
90 public void setSecond(int second) {
91     if (second < 0 || second >= 60) {
92         throw new IllegalArgumentException("second must be 0-59");
93     }
94     this.second = second;
95 }
96
97 // Get Methods
98 // get hour value
99 public int getHour() {return hour;}
100
101 // get minute value
102 public int getMinute() {return minute;}
103
104 // get second value
105 public int getSecond() {return second;}

```

Fig. 8.5 | Time2 class declaration with overloaded constructors. (Part 5 of 6.)

```

107
108 // convert to String in universal-time format (HH:MM:SS)
109 public String toUniversalString() {
110     return String.format(
111         "%02d:%02d:%02d", getHour(), getMinute(), getSecond());
112 }
113
114 // convert to String in standard-time format (H:MM:SS AM or PM)
115 public String toString() {
116     return String.format("%d:%02d:%02d %s",
117         ((getHour() == 0 || getHour() == 12) ? 12 : getHour() % 12),
118         getMinute(), getSecond(), (getHour() < 12 ? "AM" : "PM"));
119 }
120 }

```

Fig. 8.5 | Time2 class declaration with overloaded constructors. (Part 6 of 6.)

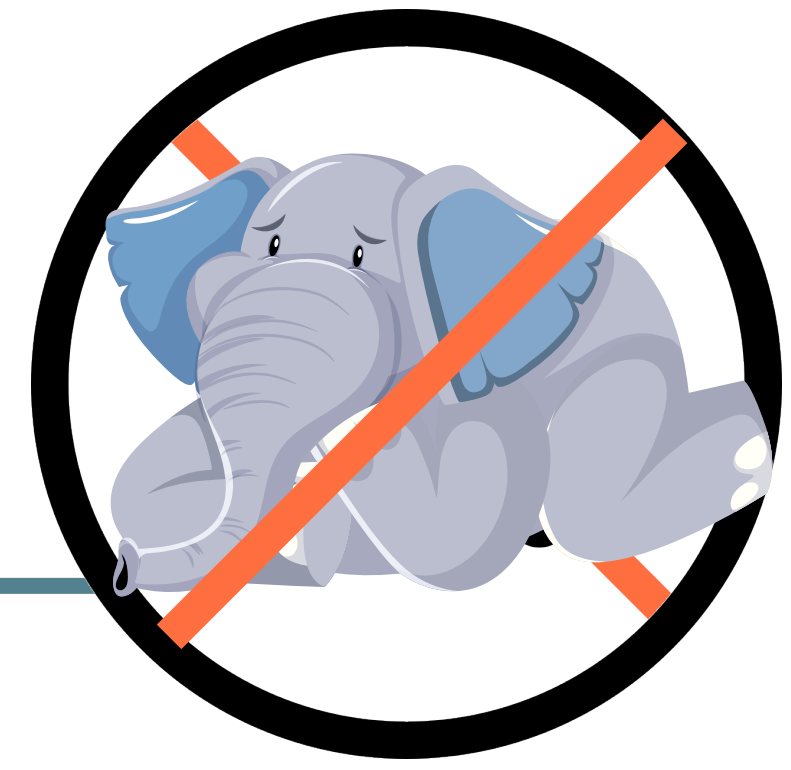
SHORT-QUESTIONS



toString() ve toUniversalString() metotları doğrudan private değişkenlere erişme yetkisine sahiptir.

Neden get metotlarını kullandık?

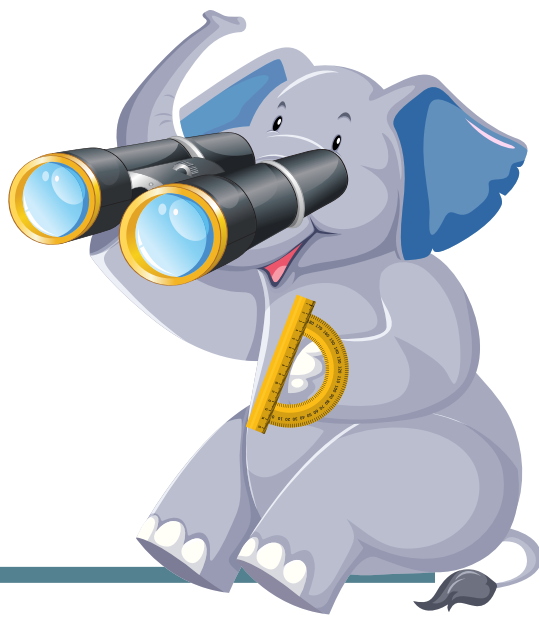
COMMON PROGRAMMING ERROR



Bir constructor gövdesinde sınıfa ait başka bir constructor çağırarak için kullanıldığında, eğer bu çağrı kurucudaki ilk ifade değilse, bu bir derleme hatasıdır.

Ayrıca, bir yöntemde, “this” ile bir constructor doğrudan çağırıldığında bu da bir derleme hatasıdır.

SOFTWARE ENGINEERING OBSERVATION



Bir sınıfın bir nesnesinin aynı sınıftaki başka bir nesneye referans olduğunda, ilk nesne ikinci nesnenin tüm verilerine ve yöntemlerine (private olanlar dahil) erişebilir.



- ▶ t1 : 3 değişken de 0 olarak atanacaktır.
- ▶ t2 : saat değeri 2, gerisi 0.
- ▶ t3 : saat 21, dakika 34, saniye 0
- ▶ t4 : saat 12, dakika 25, saniye 42
- ▶ t5 : t4 nesnesinin değerlerini alacaktır.

```
1 // Fig. 8.6: Time2Test.java
2 // Overloaded constructors used to initialize Time2 objects.
3
4 public class Time2Test {
5     public static void main(String[] args) {
6         Time2 t1 = new Time2(); // 00:00:00
7         Time2 t2 = new Time2(2); // 02:00:00
8         Time2 t3 = new Time2(21, 34); // 21:34:00
9         Time2 t4 = new Time2(12, 25, 42); // 12:25:42
10        Time2 t5 = new Time2(t4); // 12:25:42
11
12        System.out.println("Constructed with:");
13        displayTime("t1: all default arguments", t1);
14        displayTime("t2: hour specified; default minute and second", t2);
15        displayTime("t3: hour and minute specified; default second", t3);
16        displayTime("t4: hour, minute and second specified", t4);
17        displayTime("t5: Time2 object t4 specified", t5);
18    }
19 }
```

Fig. 8.6 | Overloaded constructors used to initialize Time2 objects. (Part I of 3.)



- ▶ t1 : 3 değişken de 0 olarak atanacaktır.
- ▶ t2 : saat değeri 2, gerisi 0.
- ▶ t3 : saat 21, dakika 34, saniye 0
- ▶ t4 : saat 12, dakika 25, saniye 42
- ▶ t5 : t4 nesnesinin değerlerini alacaktır.
- ▶ t6 : geçersiz değer istinası atılır.

```
1 // Fig. 8.6: Time2Test.java
2 // Overloaded constructors used to initialize Time2 objects.
3
```

```
18
19 // attempt to initialize t6 with invalid values
20 try {
21     Time2 t6 = new Time2(27, 74, 99); // invalid values
22 }
23 catch (IllegalArgumentException e) {
24     System.out.printf("%nException while initializing t6: %s%n",
25                       e.getMessage());
26 }
27 }
28
29 // displays a Time2 object in 24-hour and 12-hour formats
30 private static void displayTime(String header, Time2 t) {
31     System.out.printf("%s%n   %s%n   %s%n",
32                       header, t.toUniversalString(), t.toString());
33 }
34 }
```

```
", t2);
", t3);
```

Fig. 8.6 | Overloaded constructors used to initialize Time2 objects. (Part 2 of 3.)



- ▶ t1 : 3 değişken de 0 olarak atanacaktır.
- ▶ t2 : saat değeri 2, gerisi 0.
- ▶ t3 : saat 21, dakika 34, saniye 0
- ▶ t4 : saat 12, dakika 25, saniye 42
- ▶ t5 : t4 nesnesinin değerlerini alacaktır.
- ▶ t6 : geçersiz değer istinası atılır.

18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34

```
Constructed with:
t1: all default arguments
    00:00:00
    12:00:00 AM
t2: hour specified; default minute and second
    02:00:00
    2:00:00 AM
t3: hour and minute specified; default second
    21:34:00
    9:34:00 PM
t4: hour, minute and second specified
    12:25:42
    12:25:42 PM
t5: Time2 object t4 specified
    12:25:42
    12:25:42 PM

Exception while initializing t6: hour must be 0-23
```

Fig. 8.6 | Overloaded constructors used to initialize Time2 objects. (Part 3 of 3.)

```
// displays a Time2 object (in t4-hour and t5-minute format), t5);
private static void displayTime(String header, Time2 t) {
    System.out.printf("%s\n\t%s\n\t%s\n",
        header, t.toUniversalString(), t.toString());
}
```

Fig. 8.6 | Overloaded constructors used to initialize Time2 objects. (Part 2 of 3.)

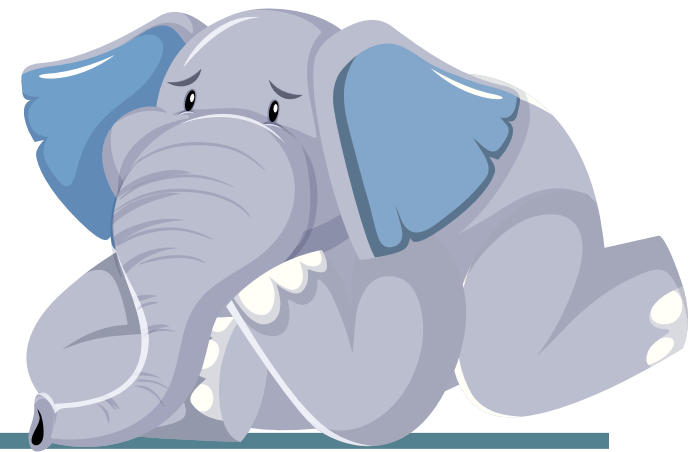


- ▶ Bir program, argümanlar olmadan çağrılan, argümansız bir constructor bildirebilir.
- ▶ Böyle bir constructor, nesneyi constructor gövdesinde belirtildiği gibi basitçe başlatır.
- ▶ Bunu yöntem çağrısı söz diziminde bir constructor gövdesindeki ilk ifade olarak kullanmak, aynı sınıftan başka bir kurucuyu çağırır. (Slayt 24)
- ▶ Argümansız constructor gövdesinde benzer bir kod tanımlamak yerine, sınıfın başka bir constructor tarafından sağlanan başlatma kodunu yeniden kullanmanın popüler yolu.
- ▶ Bir sınıfta herhangi bir constructor tanımladığınızda, derleyici varsayılan bir constructor sağlamayacaktır.



- ▶ Her sınıfta en az bir constructor olmalıdır.
- ▶ Bir sınıfın bildiriminde herhangi bir constructor sağlamazsanız, derleyici çağrıldığında hiçbir argüman almayan varsayılan bir constructor oluşturur.
- ▶ Varsayılan constructor, örnek değişkenlerini bildirimlerinde belirtilen başlangıç değerlerine veya varsayılan değerlerine başlatır (ilkel sayısal türler için 0, boolean değerleri için FALSE ve referanslar için NULL).
- ▶ Sınıfınız constructor bildirirse, derleyicinin varsayılan bir constructor oluşturmayacağını hatırlayın.
 - ▶ Bu durumda, varsayılan başlatma gerekiyorsa, argümansız bir constructor bildirmeniz gerekir.
 - ▶ Varsayılan bir constructor gibi, argümansız bir constructor boş parantezlerle çağrılır.
 - ▶ `public sınıfismi()`

ERROR PREVENTION TIP

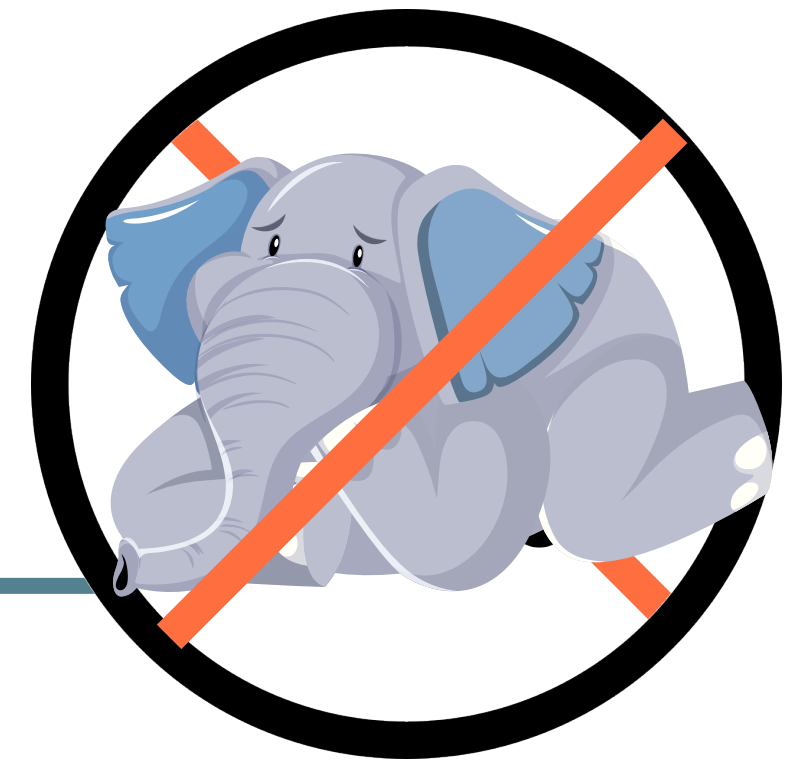


Constructor tanımına bir dönüş türü eklemediğinizden emin olun!

Java, constructor dışında sınıfın diğer metotlarının sınıfla aynı ada sahip olmasına ve dönüş türlerini belirtmesine izin verir.

Bu tür metotlar constructor değildir ve sınıfın bir nesnesi başlatıldığında çağrılmaz!

COMMON PROGRAMMING ERROR

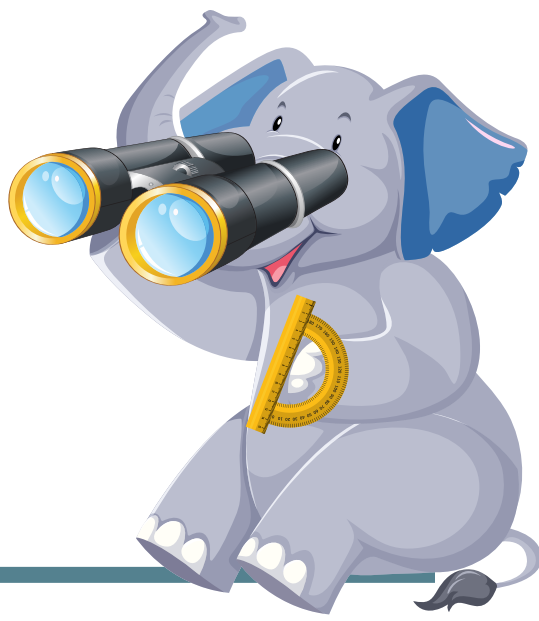


Bir program, constructor'a yanlış sayıda veya türde argüman ileterek sınıfın bir nesnesini ilklendirmeye çalışırsa, bir derleme hatası oluşur.



- ▶ Client sadece metotlar aracılığı ile bir sınıfın private alanlarını düzenleyebilir.
 - ▶ Örneğin, bir banka hesabındaki private “balance” alanı için `computeInterest()` metodu kullanılabilir.
- ▶ SET metotlarına “mutator”
 - ▶ (mutator: mutasyona neden olan şey)
- ▶ GET metotlarına “accessor” veya “query” metot ismi verilir.
 - ▶ (accessor: erişimci / query: sorgu)

SOFTWARE ENGINEERING OBSERVATION



Sınıflar hiçbir zaman “public non-constant” verilere sahip olmamalıdır.

Ancak verileri “public static final” olarak bildirmek, sabitleri sınıfınızın client ler için kullanılabilir hale getirmenizi sağlar.

Örneğin, Math sınıfı, “public static final” olarak Math.E ve Math.PI değişkenlerini sunar.



- ▶ Örnek değişkenleri private olarak bildirildiği için veri bütünlüğünün faydaları otomatik olarak takip edilmez; bu yüzden set metotlar ile geçerlilik denetimi sağlamanız gerekir.
- ▶ Accessor metotların bir türü de TRUE-FALSE sorgulaması yapan metotlardır.
- ▶ Bu metotlara “predicate methods” ismi verilir.
- ▶ Örneğin, bir listenin boş olup olmadığını sorgulamak için “isEmpty()” kullanılabilir.

GOOD PROGRAMMING PRACTICE



**Geleneksel olarak,
predicate metotların isimleri**

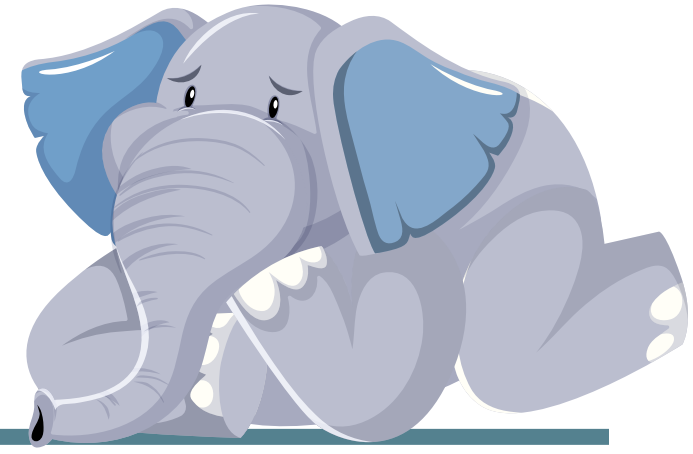
“get...”

ile değil

“is...”

ile başlar.

ERROR PREVENTION TIP



Set ve Get metotları kullanmak sürdürülebilirliği ve debug işlemlerini kolaylaştırır.

Bir değişkenin set edilmesini kontrol etmek için sadece ilgili metoda yoğunlaşırsınız.



- Bir sınıf başka bir sınıfın nesnelere üye olarak referans verebilir.
- Bu duruma “composition” veya “has-a relationship” denir.
- AlarmClock nesnesinin; şimdiki zamanı ve alarm çalacak zamanı bilmesi gerekir. Bu durumda, AlarmClock nesnesinde Time nesnesine iki referans içermesi beklenir.



HAS A

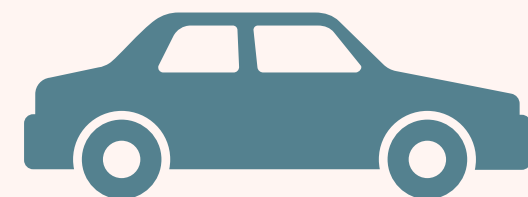


Current time

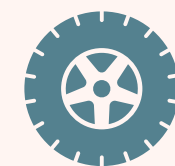


Alarm time

- Bir araba -> “has-a” -> direksiyon, fren pedalı, gaz pedalı,



HAS A





- ▶ `Date`, `Employee`, `EmployeeTest` sınıfları olsun.
- ▶ `Date`; bir tarihi temsil etmek için ay, gün ve yıl örnek değişkenlerini bildirir.
 - ▶ Gün için ilgili aya göre kontrol yapar.
 - ▶ Şubat ayı için özel kontrol yapar. (Artık yıl hesabı)
 - ▶ `toString()` ile değerleri sınıftan referans alarak (implicit), istenilen formatta gönderir.
- ▶ `Employee`; bir çalışan için, isim, doğum tarihi, işe giriş tarihi değişkenlerini bildirir.
 - ▶ Tarihler için `Date` sınıfından bir nesneyi örnek değişken olarak kullanır.
 - ▶ `toString()` ile değerleri sınıftan referans alarak (implicit), istenilen formatta gönderir.
- ▶ `EmployeeTest`; iki çalışan için bilgileri ilklendiren ve değerleri basan bir sınıftır.



```
1 // Fig. 8.7: Date.java
2 // Date class declaration.
3
4 public class Date {
5     private int month; // 1-12
6     private int day; // 1-31 based on month
7     private int year; // any year
8
9     private static final int[] daysPerMonth =
10         {0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
11 }
```

Fig. 8.7 | Date class declaration. (Part 1 of 3.)



```
1 // Fig. 8.7: Date.java
2 // Date class
3
4 public class Date {
5     private int month;
6     private int day;
7     private int year;
8
9     private static final int[] daysPerMonth = {0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
10
11
12     // constructor: confirm proper value for month and day given the year
13     public Date(int month, int day, int year) {
14         // check if month in range
15         if (month <= 0 || month > 12) {
16             throw new IllegalArgumentException(
17                 "month (" + month + ") must be 1-12");
18         }
19
20         // check if day in range for month
21         if (day <= 0 ||
22             (day > daysPerMonth[month] && !(month == 2 && day == 29))) {
23             throw new IllegalArgumentException("day (" + day +
24                 ") out-of-range for the specified month and year");
25         }
26     }
27 }
```

Fig. 8.7 | Date class declaration. (Part 2 of 3.)

Fig. 8.7 | Date class declaration. (Part 2 of 3.)



```

1 // Fig. 8.7: Date.java
2 // Date class declaration.
3
4 public class Date { // constructor:
5     private int month; // 1-12
6     private int day; // 1-31, based on month
7     private int year; // any year
8
9     private static final int[] daysPerMonth = {
10         0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 31
11     };
12
13     // check if day is valid
14     if (day <= 0 || day > daysPerMonth[month - 1])
15         throw new IllegalArgumentException("day (" + day +
16             ") out-of-range for the specified month and year");
17
18     this.month = month;
19     this.day = day;
20     this.year = year;
21
22     // return a String of the form month/day/year
23     public String toString() {
24         return String.format("%d/%d/%d", month, day, year);
25     }
26

```

Fig. 8.7 | Date class declaration. (Part 1 of 3.)

Fig. 8.7 | Date class declaration. (Part 2 of 3.)

```

27 // check for leap year if month is 2 and day is 29
28 if (month == 2 && day == 29 && !(year % 400 == 0 ||
29     (year % 4 == 0 && year % 100 != 0))) {
30     throw new IllegalArgumentException("day (" + day +
31         ") out-of-range for the specified month and year");
32 }
33
34 this.month = month;
35 this.day = day;
36 this.year = year;
37
38 System.out.printf("Date object constructor for date %s\n", this);
39 }
40
41 // return a String of the form month/day/year
42 public String toString() {
43     return String.format("%d/%d/%d", month, day, year);
44 }
45 }

```

Fig. 8.7 | Date class declaration. (Part 3 of 3.)



```
1 // Fig. 8.8: Employee.java
2 // Employee class with references to other objects.
3
4 public class Employee {
5     private String firstName;
6     private String lastName;
7     private Date birthDate;
8     private Date hireDate;
9
10    // constructor to initialize name, birth date and hire date
11    public Employee(String firstName, String lastName, Date birthDate,
12        Date hireDate) {
13        this.firstName = firstName;
14        this.lastName = lastName;
15        this.birthDate = birthDate;
16        this.hireDate = hireDate;
17    }
18
19    // convert Employee to String format
20    public String toString() {
21        return String.format("%s, %s Hired: %s Birthday: %s",
22            lastName, firstName, hireDate, birthDate);
23    }
24 }
```

Fig. 8.8 | Employee class with references to other objects.



```
1 // Fig. 8.8: Employee.java
2 // Employee class with referen
3
4 public class Employee {
5     private String firstName;
6     private String lastName;
7     private Date birthDate;
8     private Date hireDate;
9
10    // constructor to initializ
11    public Employee(String first
12        Date hireDate) {
13        this.firstName = firstNa
14        this.lastName = lastName
15        this.birthDate = birthDa
16        this.hireDate = hireDate
17    }
18
19    // convert Employee to Stri
20    public String toString() {
21        return String.format("%s
22        lastName, firstName,
23    }
24 }
```

Fig. 8.8 | Employee class with references

```
1 // Fig. 8.9: EmployeeTest.java
2 // Composition demonstration.
3
4 public class EmployeeTest {
5     public static void main(String[] args) {
6         Date birth = new Date(7, 24, 1949);
7         Date hire = new Date(3, 12, 1988);
8         Employee employee = new Employee("Bob", "Blue", birth, hire);
9
10        System.out.println(employee);
11    }
12 }
```

Date object constructor for date 7/24/1949
Date object constructor for date 3/12/1988
Blue, Bob Hired: 3/12/1988 Birthday: 7/24/1949

Fig. 8.9 | Composition demonstration.



- ▶ Temel olarak enum türü; benzersiz tanımlayıcılar olarak temsil edilen bir dizi sabiti tanımlar.
- ▶ Sınıflar gibi, tüm enum türleri referans türleridir.
- ▶ Bir enum türü, enum sabitlerinin virgülle ayrılmış bir listesi olan bir “enum bildirimi” ile bildirilir.
- ▶ Bildirim isteğe bağlı olarak constructors, alanlar ve metotlar gibi sınıfların geleneksel diğer bileşenlerini içerebilir.
- ▶ Enum bildirimleri şu kısıtlar altında bildirilir;
 - ▶ Enum sabitleri “final” olmalıdır.
 - ▶ Enum sabitleri “static” olmalıdır.
 - ▶ “new” ile enum türüne erişmek bir derleme hatasına yol açar.



- ▶ Enum sabitleri, sabit değere ihtiyaç olunan her yerde kullanılabilir.
- ▶ Switch için case etiketleri
- ▶ For loop values

```
enum Level {  
    LOW,  
    MEDIUM,  
    HIGH  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Level myVar = Level.MEDIUM;  
  
        switch(myVar) {  
            case LOW:  
                System.out.println("Low level");  
                break;  
            case MEDIUM:  
                System.out.println("Medium level");  
                break;  
            case HIGH:  
                System.out.println("High level");  
                break;  
        }  
    }  
}
```



```
1 // Fig. 8.10: Book.java
2 // Declaring an enum type with a constructor and explicit instance fields
3 // and accessors for these fields
4
5 public enum Book {
6     // declare constants of enum type
7     JHTP("Java How to Program", "2018"),
8     CHTP("C How to Program", "2016"),
9     IW3HTP("Internet & World Wide Web How to Program", "2012"),
10    CPPHTP("C++ How to Program", "2017"),
11    VBHTP("Visual Basic How to Program", "2014"),
12    CSHARPHTP("Visual C# How to Program", "2017");
13
14    // instance fields
15    private final String title; // book title
16    private final String copyrightYear; // copyright year
17
```

Fig. 8.10 | Declaring an enum type with a constructor and explicit instance fields and accessors for these fields. (Part 1 of 2.)



```
1 // Fig. 8.10: Book.java
2 // Declaring an enum
3 // and accessors for
4
5 public enum Book {
6     // declare constants
7     JHTP("Java How to
8     CHTP("C How to Pro
9     IW3HTP("Internet &
10    CPPHTP("C++ How to
11    VBHTP("Visual Basi
12    CSHARPHTP("Visual
13
14    // instance fields
15    private final Stri
16    private final Stri
17
```

Fig. 8.10 | Declaring an enum
for these fields. (Part 1 of 2.)

```
18    // enum constructor
19    Book(String title, String copyrightYear) {
20        this.title = title;
21        this.copyrightYear = copyrightYear;
22    }
23
24    // accessor for field title
25    public String getTitle() {
26        return title;
27    }
28
29    // accessor for field copyrightYear
30    public String getCopyrightYear() {
31        return copyrightYear;
32    }
33 }
```

Fig. 8.10 | Declaring an enum type with a constructor and explicit instance fields and accessors
for these fields. (Part 2 of 2.)



```
1 // Fig. 8.11: EnumTest.java
2 // Testing enum type Book.
3 import java.util.EnumSet;
4
5 public class EnumTest {
6     public static void main(String[] args) {
7         System.out.println("All books:");
8
9         // print all books in enum Book
10        for (Book book : Book.values()) {
11            System.out.printf("%-10s%-45s%s\n", book,
12                               book.getTitle(), book.getCopyrightYear());
13        }
14
15        System.out.printf("\nDisplay a range of enum constants:\n");
16
17        // print first four books
18        for (Book book : EnumSet.range(Book.JHTP, Book.CPPHTP)) {
19            System.out.printf("%-10s%-45s%s\n", book,
20                               book.getTitle(), book.getCopyrightYear());
21        }
22    }
23 }
```

Fig. 8.11 | Testing enum type Book. (Part 1 of 2.)



```

1 // Fig. 8.11: EnumTest.java
2 // Testing enum type Book.
3 import java.util.EnumSet;
4
5 public class EnumTest {
6     public static void main(String[] args) {
7         System.out.println("All books:");
8
9         // print all books in enum Book
10        for (Book book : Book.values())
11            System.out.printf("%-10s%-4s\n",
12                               book.getTitle(), book.getYear());
13    }
14
15    System.out.printf("\nDisplay a range of enum constants:");
16
17    // print first four books
18    for (Book book : EnumSet.range(Book.JHTP, Book.CPPHTP))
19        System.out.printf("%-10s%-4s\n",
20                            book.getTitle(), book.getYear());
21    }
22 }
23 }

```

Fig. 8.11 | Testing enum type Book. (Part 1 of 2.)

```

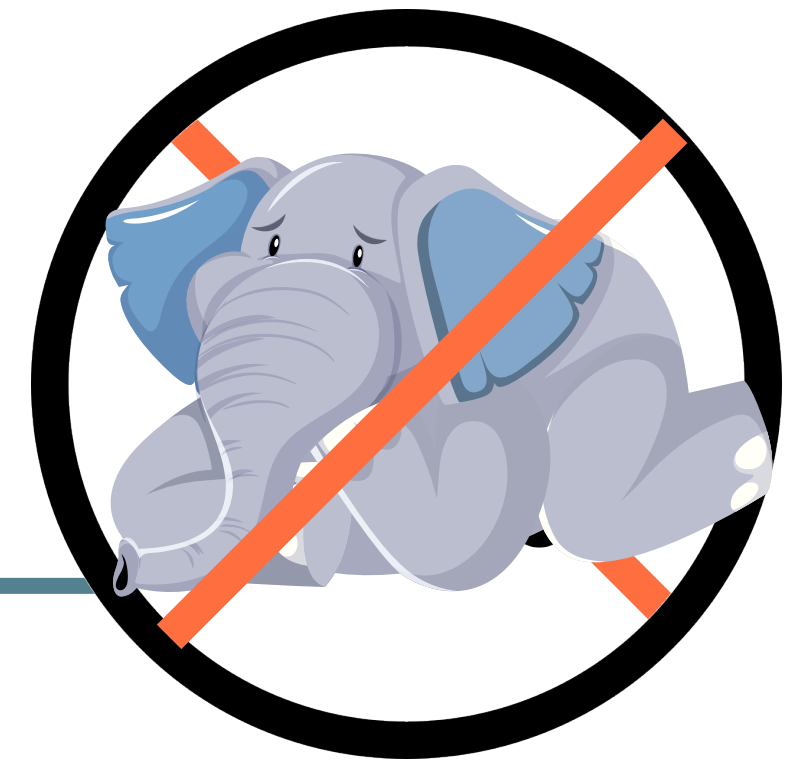
All books:
JHTP      Java How to Program      2018
CHTP      C How to Program          2016
IW3HTP    Internet & World Wide Web How to Program 2012
CPPHTP    C++ How to Program               2017
VBHTP     Visual Basic How to Program     2014
CSHARPHTP Visual C# How to Program       2017

Display a range of enum constants:
JHTP      Java How to Program      2018
CHTP      C How to Program          2016
IW3HTP    Internet & World Wide Web How to Program 2012
CPPHTP    C++ How to Program               2017

```

Fig. 8.11 | Testing enum type Book. (Part 2 of 2.)

COMMON PROGRAMMING ERROR



**Bir “enum” bildiriminde,
enum sabitleri, enum constructor, Alan ve metotlarından
sonra bildirilirse sözdizimi (syntax) hatası oluşur.**

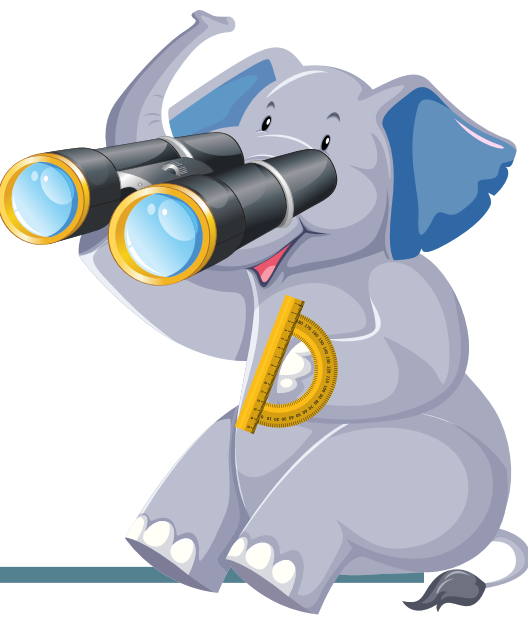


- ▶ Her nesne, bellek gibi sistem kaynaklarını kullanır.
- ▶ Nesneye artık ihtiyaç olmadığında tahsis edilen kaynağın tekrar geri verilmesi gerekir.
- ▶ Aksi takdirde “kaynak sızıntısı” oluşur. (Diğer programların kaynağı kullanmasını engelleyen olay.)
- ▶ JVM, artık kullanılmayan nesneler tarafından işgal edilen belleği geri kazanmak için otomatik çöp toplama (garbage collection) işlemi gerçekleştirir.
- ▶ Bir nesneye daha fazla referans olmadığında, nesne toplanmaya uygundur.
- ▶ Toplama, tipik olarak, JVM garbage collector çalıştırdığında gerçekleşir. (Program kapandıktan sonra)
- ▶ C ve C++ gibi diğer dillerde yaygın olan bellek sızıntıları (çünkü bu dillerde bellek otomatik olarak geri kazanılmaz) Java’ya göre daha az olasıdır.
- ▶ Bellek sızıntıları dışında kaynak sızıntıları da meydana gelebilir.
- ▶ Bir uygulama, içeriğini değiştirmek için diskteki bir dosyayı açabilir.
- ▶ Uygulama dosyayı kapatmazsa, başka bir uygulamanın dosyayı kullanabilmesi için sonlandırılmalıdır.



- ▶ Java'daki her sınıf, Object sınıfının metotlarına (package java.lang) sahiptir. (Bunlardan biri finalize())
- ▶ Finalize metodunu asla kullanmamalısınız, çünkü birçok soruna neden olabilir.
 - ▶ (bir program sonlandırılmadan önce çağrılıp çağrılmayacağı konusunda belirsizlik vardır.)
- ▶ **ÖNCELERİ:** Finalize metodunun asıl amacı, çöp toplayıcının, nesnenin hafızasını geri kazanmadan hemen önce bir nesne üzerinde sonlandırma temizliği yapmasına izin vermektir.
- ▶ **ŞİMDİ:** Programcılar bir programda artık ihtiyaç duymadıklarında kaynakları serbest bırakmak için arayabilecekleri bir yöntem sağlamak için, diskteki dosyalar gibi sistem kaynaklarını kullanan herhangi bir sınıf için daha iyi bir uygulama olarak kabul edilir.
- ▶ AutoClosable nesneleri, onları "try" ifadesiyle kullandığınızda kaynak sızıntısı olasılığını azaltır.
- ▶ Adından da anlaşılacağı gibi, bir AutoClosable nesnesi, "try" ifadesi nesneyi kullanmayı bitirdiğinde otomatik olarak kapatılır.

SOFTWARE ENGINEERING OBSERVATION



Çoğu Java API sınıfı (Scanner, dosya yazma/okuma sınıfları, vs.) kaynaklara ihtiyaç kalmadığında, onları serbest bırakmak için CLOSE veya DISPOSE gibi metotlar içerir.

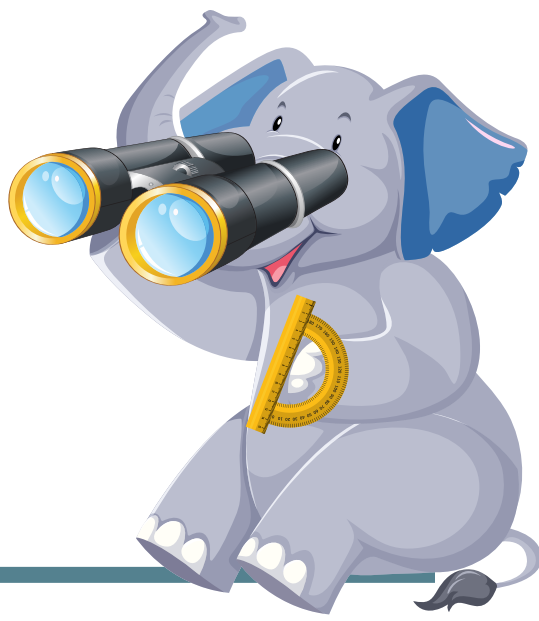


- ▶ Belirli durumlarda, belirli bir değişkenin yalnızca bir kopyası, bir sınıfın tüm nesneleri tarafından paylaşılmalıdır.
- ▶ Bu gibi durumlarda “sınıf değişkeni” adı verilen statik bir alan kullanılır. (static anahtar kelimesi ile)
- ▶ Statik bir değişken, tüm sınıfa ait bilgileri temsil eder; sınıfın tüm nesneleri aynı veri parçasını paylaşır.
- ▶ ÖRNEK;
 - ▶ Marslılar (Martians) ve diğer uzay yaratıkları olan bir oyunumuz olsun.
 - ▶ Eğer bir Marslı, yanında en az 4 başka Marslı olduğunu bilirse cesur olur ve saldırma eğilimindedir.
 - ▶ Ancak eğer 5’den az Marslı varsa, hepsi korkaklaşır :)
 - ▶ Bu durumda “MartianCount” adında bir örnek değişken olsun. Her yeni Marslı yaratıldığında bu değişken güncellensin.
 - ▶ Her bir Marslı da bu değişkenin bir kopyasından sayılarını bilsin. Güncelleme zaman alır ve hatalar oluşabilir.
 - ▶ Bu değişken “static” olursa, sınıf kapsamında bir değişkene dönüşür. Tek bir kopyayı her Marslı görür.



- ▶ Statik değişkenlerin kapsamı (scope) tüm sınıftır; sınıfın tüm metotlarında kullanılabilirler.
- ▶ Bir sınıfın “public static” üyelerine iki yolla erişilebilir;
 - ▶ Sınıfın herhangi bir nesnesine başvuru yoluyla
 - ▶ Üye adını sınıf adı ve bir nokta (.) ile niteleyerek (Math.random() veya Math.PI gibi) erişebilir.
- ▶ Bir sınıfın “private static” üyelerine yalnızca sınıfın metotları aracılığıyla istemci kodu tarafından erişilebilir.

SOFTWARE ENGINEERING OBSERVATION

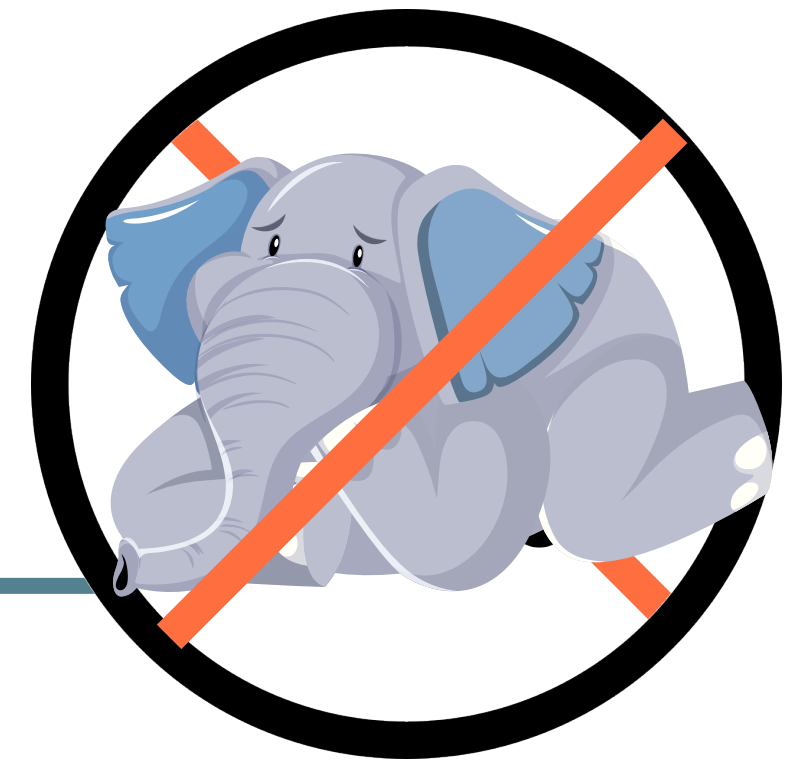


**Statik sınıf değişkenleri ve metotları, o sınıftan hiç bir nesne
örneklendirilmemiş olsa bile hayattadırlar ve
kullanılabilirler.**



- ▶ Statik sınıf üyeleri, sınıf yürütme zamanında belleğe yüklenir yüklenmez kullanılabilir hale gelir.
- ▶ Statik bir metot, bir sınıfın örnek değişkenlerine ve örnek metotlarına erişemez, ÇÜNKÜ sınıfın hiçbir nesnesi başlatılmamış olsa bile statik bir metot çağrılabilir. (Bu durum hataya sebep olur.)
- ▶ Aynı nedenle, "this" referansı statik bir metot içerisinde kullanılamaz.
- ▶ "This", sınıfın belirli bir nesnesine atıfta bulunmalıdır ve statik bir yöntem çağrıldığında, bellekte sınıfının herhangi bir nesnesi olmayabilir.
- ▶ Statik bir değişken ilklendirilmezse, derleyici ona varsayılan bir değer atar. (Örn. int için 0.)

COMMON PROGRAMMING ERROR



Bir statik metot, aynı sınıf içindeki bir örnek metodu/değişkeni sadece ismi ile çağırırsa bir derleme hatası oluşur.

Bir statik metot içinde “this” referansına başvurmak derleme hatasıdır.



```
1 // Fig. 8.12: Employee.java
2 // static variable used to maintain a count of the number of
3 // Employee objects in memory.
4
5 public class Employee {
6     private static int count = 0; // number of Employees created
7     private String firstName;
8     private String lastName;
9
10    // initialize Employee, add 1 to static count and
11    // output String indicating that constructor was called
12    public Employee(String firstName, String lastName) {
13        this.firstName = firstName;
14        this.lastName = lastName;
15
16        ++count; // increment static count of employees
17        System.out.printf("Employee constructor: %s %s; count = %d\n",
18            firstName, lastName, count);
19    }
```

Fig. 8.12 | static variable used to maintain a count of the number of Employee objects in memory. (Part 1 of 2.)



```
1 // Fig. 8.12: Employee.java
2 // static variable used to maintain a count of the number of
3 // Employee objects in memo
4
5 public class Employee {
6     private static int count
7     private String firstName
8     private String lastName;
9
10    // initialize Employee,
11    // output String indicat
12    public Employee(String f
13        this.firstName = firs
14        this.lastName = lastN
15
16        ++count; // incremen
17        System.out.printf("Em
18            firstName, lastNam
19    }
```

Fig. 8.12 | static variable used to m
memory. (Part 1 of 2.)

```
20
21    // get first name
22    public String getFirstName() {
23        return firstName;
24    }
25
26    // get last name
27    public String getLastName() {
28        return lastName;
29    }
30
31    // static method to get static count value
32    public static int getCount() {
33        return count;
34    }
35 }
```

Fig. 8.12 | static variable used to maintain a count of the number of Employee objects in
memory. (Part 2 of 2.)

GOOD PROGRAMMING PRACTICE



Çağrılan yöntemin statik bir yöntem olduğunu vurgulamak için sınıf adını ve noktayı (.) kullanarak statik yöntemi çağırın.



- ▶ Java'daki String nesneleri değişmezdir (immutable); oluşturulduktan sonra değiştirilemezler.
 - ▶ Bu nedenle, bir String nesnesine birçok referans vermek güvenlidir.
 - ▶ Bu, normalde Java'daki diğer sınıfların çoğu için geçerli değildir.
- ▶ String nesneleri değişmezse, String nesnelerini birleştirmek için neden + ve += operatörlerini kullanabileceğimizi merak edebilirsiniz.
- ▶ String-concatenation aslında, birleştirilmiş değerleri içeren yeni bir String nesnesiyle sonuçlanır; orijinal String nesneleri değiştirilmez. :)



```
1 // Fig. 8.13: EmployeeTest.java
2 // static member demonstration.
3
4 public class EmployeeTest {
5     public static void main(String[] args) {
6         // show that count is 0 before creating Employees
7         System.out.printf("Employees before instantiation: %d\n",
8             Employee.getCount());
9
10        // create two Employees; count should be 2
11        Employee e1 = new Employee("Susan", "Baker");
12        Employee e2 = new Employee("Bob", "Blue");
13
14        // show that count is 2 after creating two Employees
15        System.out.printf("\nEmployees after instantiation:\n");
16        System.out.printf("via e1.getCount(): %d\n", e1.getCount());
17        System.out.printf("via e2.getCount(): %d\n", e2.getCount());
18        System.out.printf("via Employee.getCount(): %d\n",
19            Employee.getCount());
```

Fig. 8.13 | static member demonstration. (Part 1 of 2.)



```
1 // Fig. 8.13: EmployeeTest.java
2 // static member demonstration.
3
4 public class EmployeeTest {
5     public static void main(String
6         // show that count is 0 be
7         System.out.printf("Employee
8             Employee.getCount());
9
10        // create two Employees; c
11        Employee e1 = new Employee
12        Employee e2 = new Employee
13
14        // show that count is 2 af
15        System.out.printf("%nEmplo
16        System.out.printf("via e1.
17        System.out.printf("via e2.
18        System.out.printf("via Emp
19            Employee.getCount());
```

Fig. 8.13 | static member demonstration. (

```
20
21        // get names of Employees
22        System.out.printf("%nEmployee 1: %s %s\nEmployee 2: %s %s\n",
23            e1.getFirstName(), e1.getLastName(),
24            e2.getFirstName(), e2.getLastName());
25    }
26 }
```

Employees before instantiation: 0
Employee constructor: Susan Baker; count = 1
Employee constructor: Bob Blue; count = 2

Employees after instantiation:
via e1.getCount(): 2
via e2.getCount(): 2
via Employee.getCount(): 2

Employee 1: Susan Baker
Employee 2: Bob Blue

Fig. 8.13 | static member demonstration. (Part 2 of 2.)

STATIC IMPORT



- ▶ “static import” bir sınıfa ait statik üyelerin import edilmesini sağlar.
- ▶ `import static packageName.ClassName.staticMemberName` (sadece belirli bir tanesini seçer.)
- ▶ `import static packageName.ClassName.*` (tüm static üyeleri seçer.)
- ▶ `packageName` sınıfın paketinin ismini, `ClassName` ise sınıfın ismini verir.
- ▶ `static import SADECE` static üyeleri getirir. (asıl “import” işleminden farklı)

```
1 // Fig. 8.14: StaticImportTest.java
2 // Static import of Math class methods.
3 import static java.lang.Math.*;
4
5 public class StaticImportTest {
6     public static void main(String[] args) {
7         System.out.printf("sqrt(900.0) = %.1f\n", sqrt(900.0));
8         System.out.printf("ceil(-9.8) = %.1f\n", ceil(-9.8));
9         System.out.printf("E = %f\n", E);
10        System.out.printf("PI = %f\n", PI);
11    }
12 }
```

```
sqrt(900.0) = 30.0
ceil(-9.8) = -9.0
E = 2.718282
PI = 3.141593
```

Fig. 8.14 | static import of Math class methods.

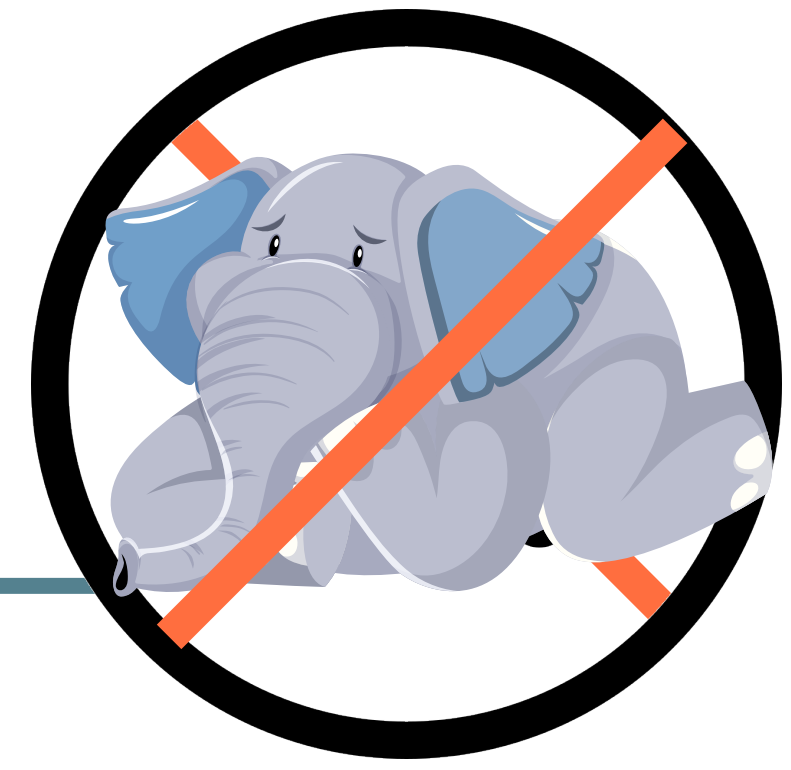


- ▶ En az ayrıcalık ilkesi, iyi bir yazılım mühendisliğinin temelidir.
- ▶ Kod, yalnızca belirlenmiş görevini yerine getirmek için ihtiyaç duyduğu ayrıcalık ve erişim miktarına verilmelidir, daha fazlası değil.
- ▶ Kodun yanlışlıkla (veya kötü niyetli olarak) değişken değerleri değiştirmesini ve erişilmemesi gereken metotların çağırmasını önleyerek programlarınızı daha sağlam hale getirir.
- ▶ “final” anahtar kelimesi, bir değişkenin değiştirilemez olduğunu (yani bir sabit olduğunu) ve onu değiştirmeye yönelik herhangi bir girişimin bir hata olduğunu belirtir.
- ▶ `private final int INCREMENT;`
- ▶ `int` türünde bir final (sabit) örnek değişkeni `INCREMENT` bildirir.



- ▶ final değişkenleri, bildirildiğinde veya sınıfın her bir nesnesinin farklı bir değere sahip olması için sınıfın her bir constructor u tarafından ilklendirilebilir.
- ▶ Bir sınıf birden çok constructor sağlıyorsa, her birinin her bir final değişkeni ilklendirmesi gerekir.
- ▶ final değişken, ilklendirildikten sonra atama ile değiştirilemez.
- ▶ final değişken ilklendirilmezse, bir derleme hatası oluşur.

COMMON PROGRAMMING ERROR



İklendirildikten sonra bir final değişkeni düzenleme girişimi bir derleme hatasına yol açar.

SOFTWARE ENGINEERING OBSERVATION

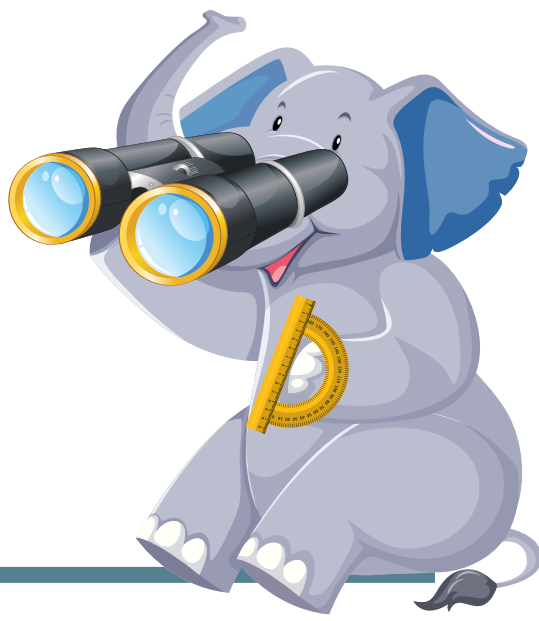
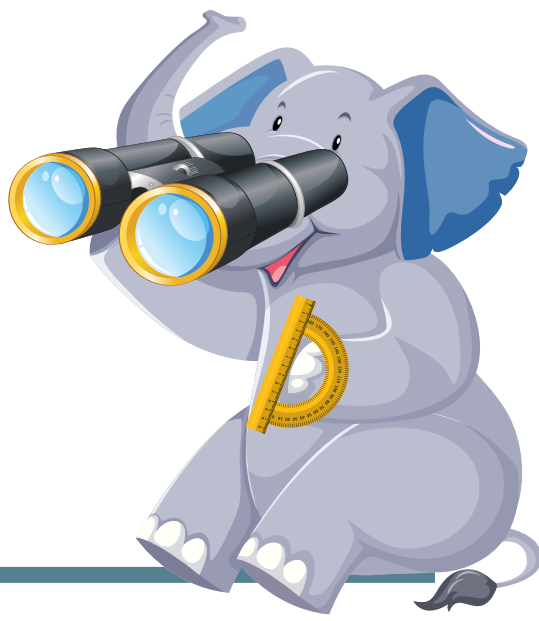


Fig 8.8 örneğindeki firstName, lastName, birthDate and hireDate ilklendirildikten sonra değişecek değerler değildir. Bu yüzden onları “final” yapmak doğru olacaktır.

Bu kullanım ilerde programı debug etmek ve hatalara karşı korumak açısından oldukça faydalı olacaktır.

SOFTWARE ENGINEERING OBSERVATION



“final” alan, eğer ilklendirildikten sonra tüm diğer nesneler için aynı değeri alacak ise, o zaman o değişken “static” de olmalıdır.

Değeri hiç değişmediği için, her bir nesne için ayrı kopyasının tutulmasına gerek yoktur.

Değişkeni static yaparak onu paylaşılabilen bir final alan yapmış oluruz.



- Bir değişken veya metod için bir erişim denetleyici (private, public, protected) yok ise, bu durumda o değişken “paket erişimi” ne sahip olur. (Bölüm 4 de tekrar tartışılacak.)
- Aynı paket içindeki diğer sınıflardan erişim sağlanabilir.
- Paket erişimi çok kullanılmayan bir yöntemdir.

```
1 // Fig. 8.15: PackageDataTest.java
2 // Package-access members of a class are accessible by other classes
3 // in the same package.
4
5 public class PackageDataTest {
6     public static void main(String[] args) {
7         PackageData packageData = new PackageData();
8
9         // output String representation of packageData
10        System.out.printf("After instantiation:%n%s%n", packageData);
11
12        // change package access data in packageData object
13        packageData.number = 77;
14        packageData.string = "Goodbye";
15
16        // output String representation of packageData
17        System.out.printf("%nAfter changing values:%n%s%n", packageData);
18    }
19 }
```

Fig. 8.15 | Package-access members of a class are accessible by other classes in the same package. (Part 1 of 2.)

```
20
21 // class with package access instance variables
22 class PackageData {
23     int number = 0; // package-access instance variable
24     String string = "Hello"; // package-access instance variable
25
26     // return PackageData object String representation
27     public String toString() {
28         return String.format("number: %d; string: %s", number, string);
29     }
30 }
```

After instantiation:
number: 0; string: Hello

After changing values:
number: 77; string: Goodbye

Fig. 8.15 | Package-access members of a class are accessible by other classes in the same package. (Part 2 of 2.)